



Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University

Community Structures and Modularity

CSC1111
Building Complex Computational Models

Dr. Marija Bezbradica

What is a community?

- Very important in modeling and understanding of complex systems.
- In order to understand the properties of a network we sometimes have to understand its mesoscopic properties.
- Microscopic (e.g., degrees or clustering coefficients) and macroscopic properties (e.g., density, characteristic path length) are not able to tell us how a network is organised at spatial scales, intermediate between those two extremes
- A set of nodes with frequent **interaction** (e.g. group, subgroup, cluster, module)

Community: A set of nodes that are connected more densely to each other than to the rest of the network. Communities may or may not overlap with each other, depending on their definitions.

Modularity: The extent to which a network is organised into multiple communities.

What is a community?

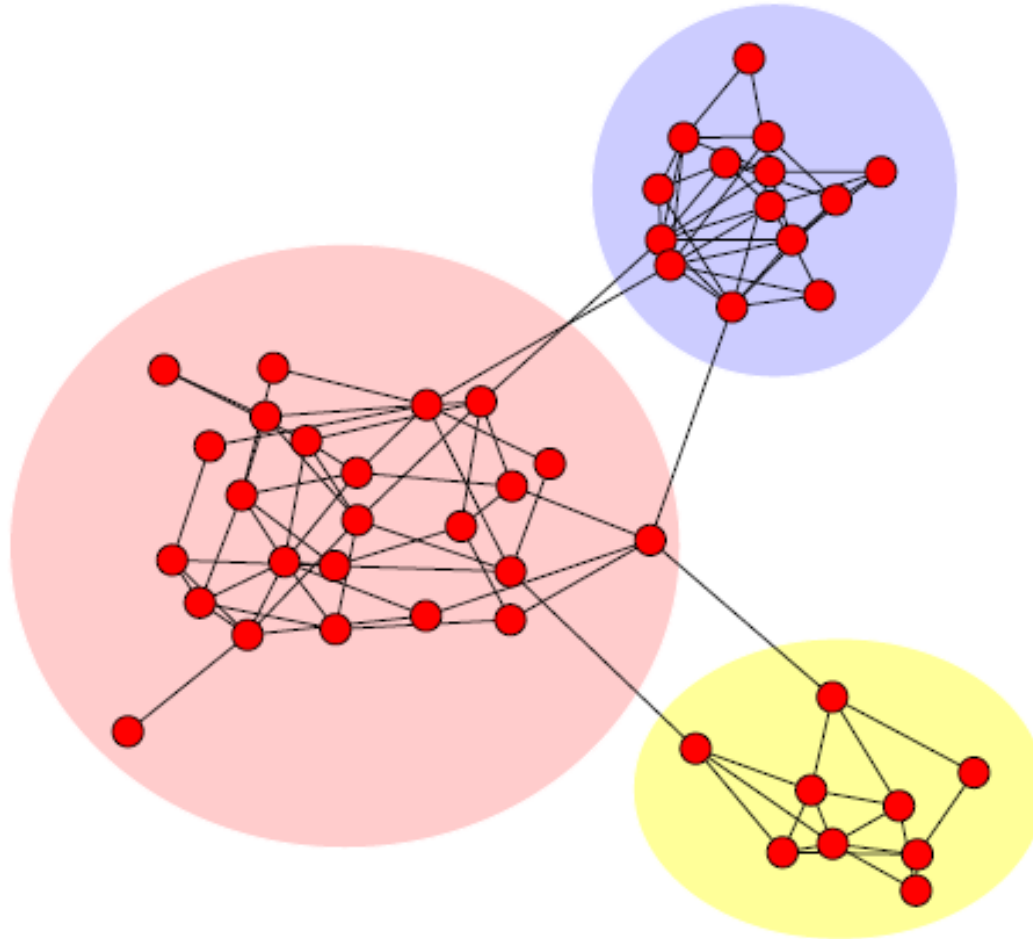


Figure: Example of communities in a network

What is a community structure?

- A network has a community structure if the nodes of the network can be **easily grouped** into (potentially overlapping) sets of nodes such that each set of nodes is densely connected internally.
- In non-overlapping communities, the network divides naturally into groups of nodes with **dense** connections **internally** and **sparser** connections **between** groups.
- Overlapping communities are also allowed.
- The pairs of nodes are more likely to be connected if they are **both members of the same community(ies)**, and less likely to be connected if they do not share communities.

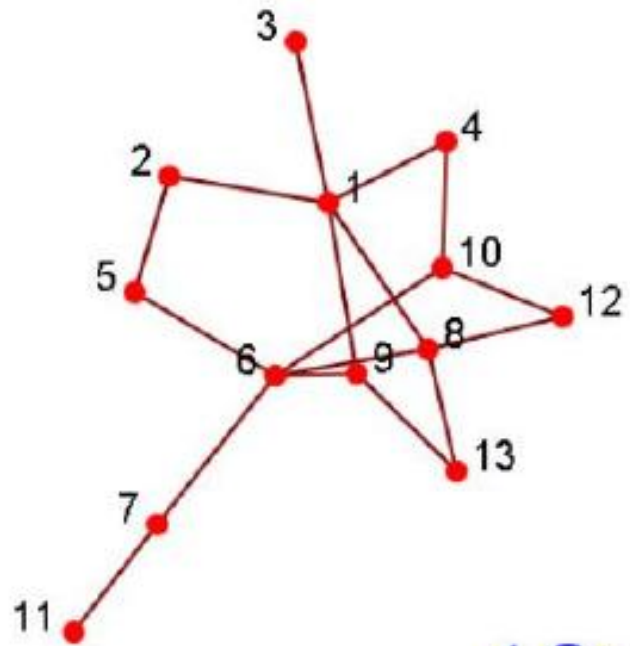
What is a community detection?

- **Detecting** 'communities' within a network
- A clustering technique can be applied to the graph to detect the communities with similar properties and behaviours so that they can be grouped
- Given: A social network
- Output: Community membership of participants (e.g. grouping, clustering, finding cohesive subgroups)
- Generally, a graph is constructed based on some algorithm

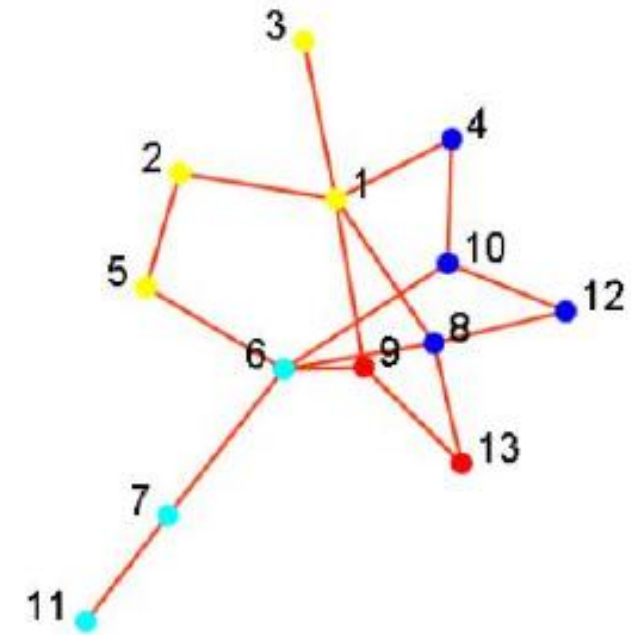
Applications:

- Understanding the interactions between people
- Visualising and navigating very large networks
- Forming basis for other tasks such as data mining

Visualisation after grouping

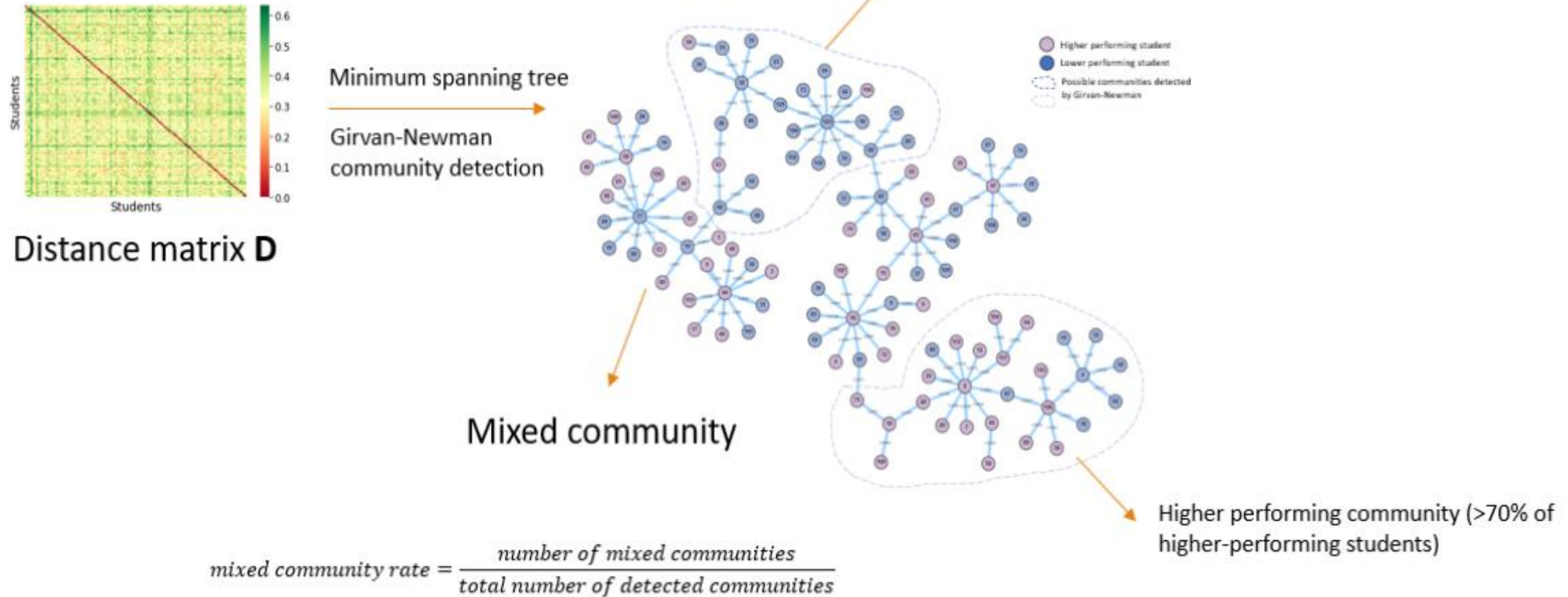


4 Groups:
 $\{1,2,3,5\}$
 $\{4,8,10,12\}$
 $\{6,7,11\}$
 $\{9,13\}$



(Nodes colored by
Community Membership)

Graph construction



We want less mixed communities and more lower/higher-performing communities

Algorithms for finding communities

- Finding communities within a network is not a trivial task.
- The number of communities, if any, within the network is typically unknown and the communities are often of unequal size and/or density.
- There are **several methods** for community finding that have been developed and employed, with varying levels of success.
- Main algorithms include Louvain, Girvan-Newman, random walk...

Community detection algorithms:

The Louvain method

- Proposed by Vincent Blondel et al. in 2008

Very fast, efficient **heuristic** algorithm that maximizes the modularity of nonoverlapping community structure through an **iterative**, hierarchical **optimisation** process.

- The **modularity** of a given set of communities in a network is defined as follows:

$$Q = \frac{|E_{\text{in}}| - \langle |E_{\text{in}}| \rangle}{|E|}$$

where,

$|E_{\text{in}}|$ - the number of within-community edges (i.e. those that do not cross boundaries between communities)

$\langle |E_{\text{in}}| \rangle$ - the expected number of within-community edges if the topology were purely random

$|E|$ - the number of edges

- The subtraction of $\langle |E_{\text{in}}| \rangle$ on the numerator penalises trivial community structure, such as considering the entire network a single community that would trivially maximise $|E_{\text{in}}|$.

The Louvain method:

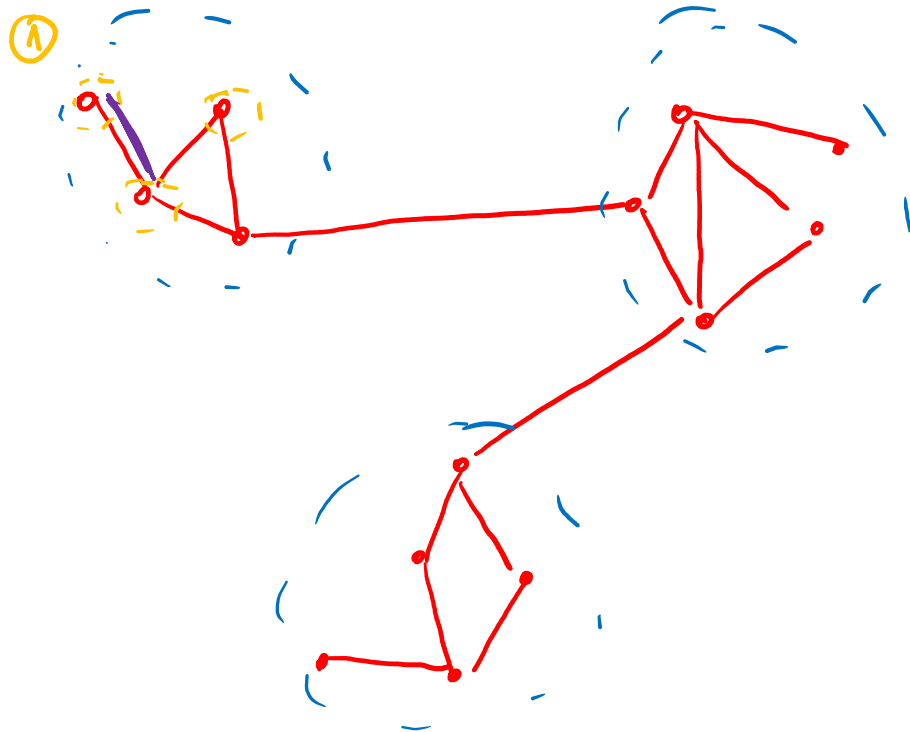
- The Louvain method finds the optimal community structure that **maximizes the modularity** in the following steps:
 1. - Initially, **each node is assigned to its own community** where the node itself is the only community member.
 - Therefore, **the number of initial communities equals the number of nodes**.
 2. - Each node **considers each of its neighbours** and evaluates whether joining to the neighbour's community would increase the modularity of the community structure.
 - After evaluating all the neighbours, it will **join the community** of the neighbour that achieves the **maximal modularity increase** (only if the change is positive; otherwise, the node will remain in its own community).
 - This will be repeatedly applied for all nodes until no more positive gain is achievable.
 3. - The result of Step 2 is converted to a **new meta-network**, by **aggregating nodes that belonged to a single community into a meta-node**, representing edges that existed within each community as the weight of a self-loop attached to the meta-node, and representing edges that existed between communities as the weights of meta-edges that connect meta-nodes.
 4. - The above two steps are repeated until no more modularity improvement is possible.

Essentially, first, small communities are found by optimising modularity locally on all nodes, then each small community is grouped into one node and the first step is repeated.

The Louvain method:

- This is a parameter free methods.
- We do not have to specify the number of communities or the criteria to stop the algorithm.
- We only need to provide network topology data, and the algorithm **heuristically** finds the community structure that is close to optimal in achieving the highest modularity.

The Louvain method:



$N_{\text{comm}} = N_{\text{nodes}} = 14$ → edges between nodes inside community

$$Q = \frac{E_{\text{in}} - \langle E_{\text{in}} \rangle}{E}$$

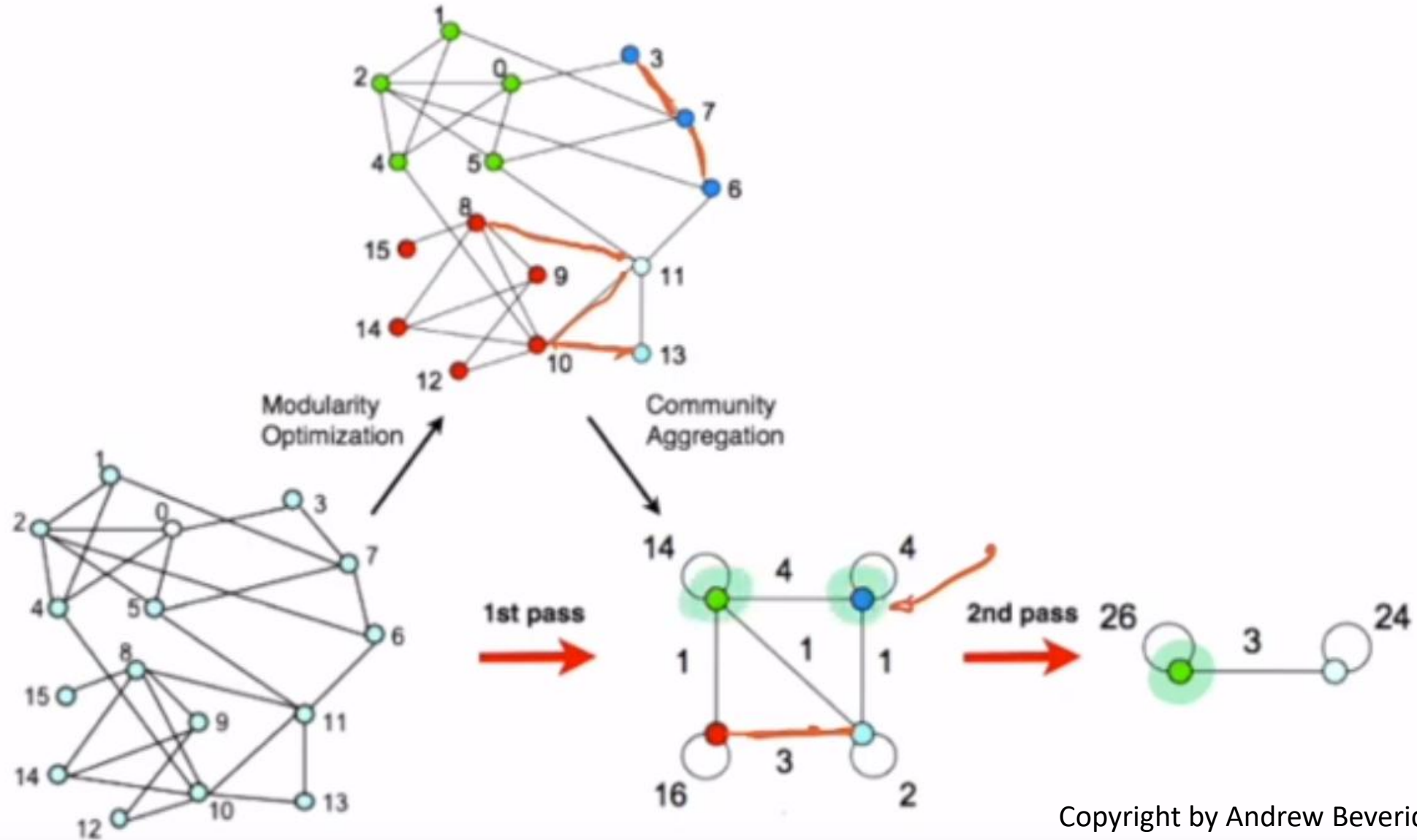
yellow $\rightarrow Q \sim \frac{0}{16} \sim 0$

purple $\rightarrow Q \sim \frac{1}{16} > 0 \Rightarrow \text{accept} \dots$

The Louvain method:

- In other words, Louvain method can be understood through two main phases (modularity optimisation and community aggregation):
 - Start with every node in its own community
 - **Phase 1: Modularity Optimization**
 - Order the nodes and do the following for each i
 - Move i to the community of neighbor j that leads to maximum ΔQ .
 - If all $\Delta Q < 0$ then i remains in its current community.
 - Repeatedly cycle through all nodes until $\Delta Q = 0$
 - **Phase 2: Community Aggregation**
 - Create a weighted network of communities from Phase 1.
 - Nodes = communities from Phase 1
 - Edge weights = sum of weights of edges between communities
 - Edges within a community become 2 self-loops
 - **Repeat:** Apply Phase 1/Phase 2 to resulting network, and so on until $\Delta Q = 0$.

The Louvain method:



NetworkX (lab work)

- There are several ways to construct networks.
- Here, we look at Python module called NetworkX for network modeling and analysis.
- All its functions are written in Python, and the source codes are all available on the website, so you can check the details of algorithms and even modify the codes if you want.
- NetworkX uses Python's plain "dictionaries" to store information about networks, which are fully accessible and flexible to store additional information you want to add.
- This property is particularly important when we implement simulation models of dynamical networks.
- The documentation for NetworkX is available online at <http://networkx.github.io>

Note: If you are using Anaconda, NetworkX is already installed.

In not, you can install with *pip install networkx*

NetworkX (lab work) - Zachari's Karate Club graph

- Famous classic example of social networks reported by Wayne Zachary in the 1970s.
- It is a network of friendships among 34 members of a karate club at a U.S. university.
- Look at the third graph in this example(Sayama code): in Loop, week 12, check [net-networkx-test2.py](#)
- Let's visualise this example:

Option 1) **call *edges()*** function on graph and you will get a list of edges in the graph:

```
>>> g3.edges()
```

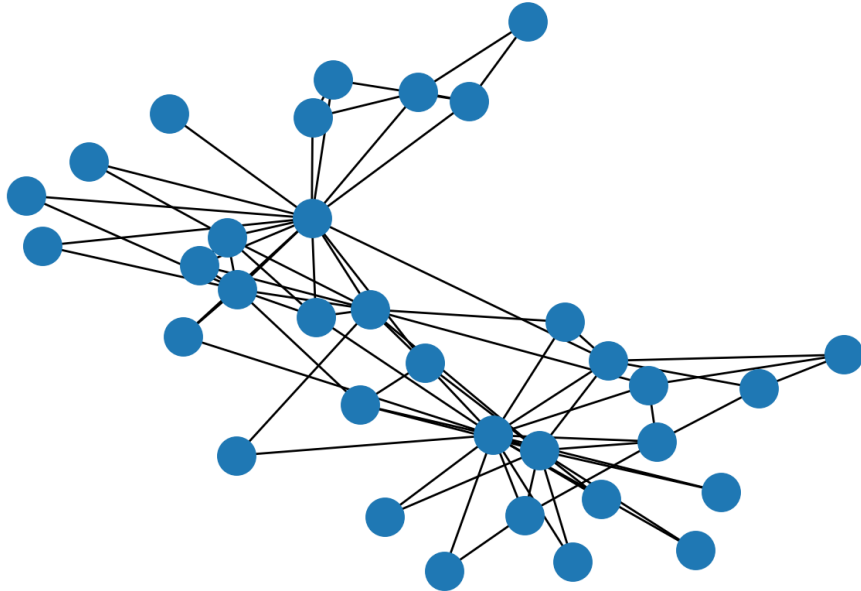
- You should get something like:

```
[(0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (0, 8), (0, 10),  
(0, 11), (0, 12), (0, 13), (0, 17), (0, 19), (0, 21), (0, 31), (1, 2),  
(1, 3), (1, 7), (1, 13), (1, 17), (1, 19), (1, 21), (1, 30), (2, 3),  
(2, 32), (2, 7), (2, 8), (2, 9), (2, 13), (2, 27), (2, 28), (3, 7)]
```

NetworkX (lab work) - Zachari's Karate Club graph

- Option 2) **visualise graph** - The simplest way is to use **NetworkX's *draw* function**:
- In Loop, week 12, check: [net-karate-club-visualization.py](#)

And you should get



Visual output of Zachary's Karate Club graph is visualized.

Your result may not look like this because the spring layout algorithm uses random initial positions by default.

- For different visualisation options check and run [network-drawing-options.py](#) (run this to check nodes' numbers as you will need it for the next slide)

NetworkX (lab work) - Zachari's Karate Club + Louvain method

- NetworkX does not have this Louvain method as a built-in function, but its Python implementation has been developed and released freely by Thomas Aynaud, which is available from <http://perso.crans.org/aynaud/communities/>

pip install python-louvain

- Once you install it, a new community module becomes available in Python. Here is an example:

```
>>> import networkx as nx
```

```
>>> import community as comm
```

```
>>> g = nx.karate_club_graph()
```

```
>>> bp = comm.best_partition(g)
```

```
>>> bp
```

```
{0: 0, 1: 0, 2: 0, 3: 0, 4: 1, 5: 1, 6: 1, 7: 0, 8: 2, 9: 0, 10: 1,
```

```
11: 0, 12: 0, 13: 0, 14: 2, 15: 2, 16: 1, 17: 0, 18: 2, 19: 0,
```

```
20: 2, 21: 0, 22: 2, 23: 3, 24: 3, 25: 3, 26: 2, 27: 3, 28: 3,
```

```
29: 2, 30: 2, 31: 3, 32: 2, 33: 2}
```

```
>>> comm.modularity(bp, g)
```

```
0.4188034188034188
```

Two important functions in the community module are tested.

- The first one is `best_partition`, which generates community structure using the Louvain method.

The result is given as a dictionary where keys and values are node IDs and community IDs, respectively.

- The second function is `modularity`, which receives community structure and a network and returns the modularity value achieved by the given communities.

NetworkX (lab work) - Louvan method

- **Ex.12.1.** Visualise the community structure in the Karate Club graph using the community IDs as the colors of the nodes.
- **Ex.12.2** Import a large network data set of your choice from Mark Newman's Network Data website: <http://www-personal.umich.edu/~mejn/netdata/>
- Detect its community structure using the Louvain method and visualise it if possible.
- **Ex.12.3** Do a quick online literature search for other community detection algorithms (e.g., Girvan-Newman method, random walk method, etc.). Choose one of them and read the literature to learn how it works. If the software is available, try it yourself on the Karate Club graph or any other network and see how the result differs from that of the Louvain method.